Rapport
Projet avancé en apprentissage automatique
IFT - 3710

Cédric Guévremont
Matricule : 20266532
cedric.guevremont@umontreal.ca

Rima Boujenane
Matricule : 20235550
rima.boujenane@umontreal.ca

Abdelghafour Rahmouni
Matricule : 20246224
abdel.ghafour.rahmouni@umontreal.ca

1 Project Description

1.1 Context

In this project, we consider a machine learning agent tasked with managing a portfolio composed of a limited set of financial assets and an initial capital. At each time step, the agent observes the state of the market from historical data (returns, volatility, technical indicators) and can perform elementary actions on these assets: buy, sell, or hold. These actions modify the portfolio allocation and generate a reward depending on performance and risk.
This framework follows a reinforcement learning approach oriented toward responsible risk management, aiming to limit excessive speculative behavior and promote stable long-term investment strategies, within a broader perspective of responsible use of machine learning methods.

1.2 Objectives

The objective of this project is to experiment with the use of recurrent neural networks (RNN/LSTM) combined with reinforcement learning, in order to evaluate to what extent these approaches make it possible to reduce the overall risk of a financial portfolio in a sequential decision-making context.

More broadly, this work opens the possibility of examining the transferability of the proposed framework to other domains based on time series and sequential decisions, such as energy, logistics, or resource management.

From this perspective, the project notably considers as exploration axes the minimization of portfolio volatility over the investment horizon, the reduction of maximum drawdowns during phases of market stress, the learning of buy, sell, and hold decisions that are coherent over time, limiting excessive reallocations, as well as the analysis of the influence of reward functions oriented toward risk control and stability on the learned allocation policies.

1.3 Planned Methods

- **Observations:** Time windows of historical financial data (returns, volatility, technical indicators) extracted via the Yahoo Finance API.
- **Modeling:** State encoder based on recurrent neural networks of type LSTM, coupled with a deep reinforcement learning agent of type Actor-Critic (PPO or DDPG).

- **Reward Function:** Combination of portfolio return and penalties related to risk (volatility) and excessive reallocations.
- **Evaluation:** Temporal backtesting with comparison to a benchmark index such as the S&P 500.

1.4 Expected Results

The expected results include obtaining a positive overall return over the test horizon, indicating that the agent learns an effective management strategy rather than a trivial behavior. It is also expected that the learned policy produces a relatively stable return, characterized by controlled volatility and limited losses during stress periods. Performance should be at a level comparable to that of the market, such as the S&P 500, while offering improved risk control and demonstrating that the agent has learned a coherent allocation strategy rather than remaining inactive.

2 Related Works

2.1 Foundational Financial Theories

Traditional portfolio optimization methods, such as Markowitz's Modern Portfolio Theory (MPT) [**?**], have been the industry standard for decades to balance risk and return through diversification. While effective in theory, MPT relies on static rules that often struggle to keep up with the chaotic, fast-moving reality of today's markets.

With the rise of machine learning, new approaches have emerged to tackle these limitations. **?** proposed integrating Deep Reinforcement Learning (DRL) with Technical Analysis and Modern Portfolio Theory constraints. Unlike traditional supervised learning models that merely predict prices, DRL agents learn sequential decision-making policies directly through interaction with the market environment. This transition from static allocation to dynamic portfolio management is emphasized in the comprehensive survey by **?**, which traces the evolution of reinforcement learning techniques in quantitative finance.

2.2 Sequential Modeling and Temporal Dependencies

Addressing the partial observability of financial markets, **?** introduced a hybrid recurrent reinforcement learning framework integrating RNN-LSTM architectures within a Q-learning setup. Their approach enables agents to maintain memory of historical market states, capturing temporal dependencies that are critical in stochastic and non-stationary environments.

2.3 The Evolution of Deep Reinforcement Learning in Finance

Building upon classical frameworks, **?** combined DRL with MPT constraints using tensor decomposition techniques to manage high-dimensional financial data within structured risk frameworks. Similarly, **?** proposed a CNN-based reinforcement learning framework that automates feature extraction from multi-channel price representations, reducing reliance on handcrafted indicators and improving scalability.

2.4 Actor-Critic Frameworks

Modern portfolio optimization strategies predominantly rely on the Actor-Critic architecture, which simultaneously updates a policy network (Actor) and a value function estimator (Critic). As implemented in the FinRL framework [**?**], in the ensemble strategy proposed by **?**, and discussed in **?**, actor-critic methods provide improved stability and continuous control capabilities.

- **DDPG** (Deep Deterministic Policy Gradient) enables continuous portfolio weight allocation and deterministic policy updates.
- **A2C** (Advantage Actor-Critic) reduces gradient variance using an advantage function, improving robustness in volatile markets.
- **PPO** (Proximal Policy Optimization) stabilizes training by constraining policy updates through clipped objective functions.

2.5 Alternative Data and Future Directions

Recent work incorporates alternative data streams to enrich state representations. **?** integrated financial news sentiment using NLP models such as FinBERT into DRL-based portfolio optimization. Emerging research by **?** explores quantum-enhanced reinforcement learning combined with LSTM forecasting signals to improve convergence stability and performance in noisy financial systems.

2.6 Risk-Aware Rewards and Responsible Management

To ensure robust and responsible financial decision-making, recent literature emphasizes risk-aware reward functions. Both the FinRL framework [**?**] and the ensemble strategy proposed by **?** incorporate market frictions such as transaction costs and financial turbulence indices. These mechanisms penalize excessive speculation and enable agents to survive extreme market conditions, including financial crashes.

3 Developed Methods and Experimental Plan

This section details the methodology for our dynamic portfolio management system, formulating asset allocation as a sequential decision-making process solved by a Proximal Policy Optimization (PPO) agent. We outline the custom environment, the multimodal Actor-Critic architecture, the risk-aware reward shaping, and the training procedure, concluding with preliminary baseline comparisons and future directions.

3.1 Environment and Data Pipeline

The portfolio management problem is formulated as a sequential decision-making task within a custom environment compatible with the `Gymnasium` interface. The environment simulates the evolution of a portfolio composed of 11 highly capitalized financial assets (MSFT, GOOG, AMZN, BRK-B, WMT, COST, JPM, V, LLY, PG, GLD).

Historical market data covering the period from January 2010 to February 2026 are retrieved via the Yahoo Finance API and processed through a dedicated data pipeline. To ensure chronological integrity and prevent data leakage, the data undergoes a strict time-series split.

At each time step t, the agent observes the state of the market and the current portfolio allocation. The observation is composed of two main components represented in a dictionary: `market_history`: a matrix of dimension $(W \times N)$ containing historical price information for the $N = 11$ assets over a rolling window of the last $W = 60$ time steps, and `portfolio_state`: a vector describing the current allocation weights of the portfolio.

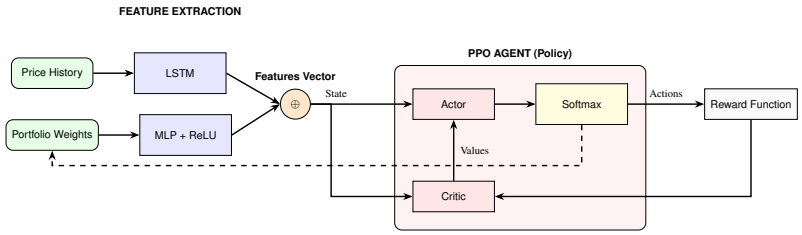
3.2 Model Architecture

The learning agent is implemented using the **Proximal Policy Optimization (PPO)** algorithm, a widely used Actor-Critic method known for its training stability and strong empirical performance in continuous control tasks.

To process the structured multimodal observation space, we implement a custom feature extractor (`CustomCombinedExtractor`) composed of two parallel streams:

- a **Long Short-Term Memory (LSTM)** network (configured with 2 layers and a hidden size of 128) that processes the 60-day historical market data to capture temporal dependencies and momentum,
- a **Multi-Layer Perceptron (MLP)** with a ReLU activation function that processes the static current portfolio allocation state.

The final hidden state of the LSTM is concatenated with the MLP output to produce a combined feature representation. This holistic state vector is then fed into the PPO agent's Actor-Critic architecture. The policy network (Actor) determines the optimal allocation, passing its output through a **Softmax** layer to ensure the resulting actions represent valid portfolio weights.



3.3 Reward Function

The agent aims to maximize the net portfolio return while minimizing rebalancing costs. At each step t, the reward R_t is defined as:

$$R_t = \underbrace{\sum_{i=1}^{n} w_{i,t} \left(\frac{p_{i,t} - p_{i,t-1}}{p_{i,t-1}} \right)}_{\text{Portfolio Return}} - \underbrace{\lambda \sum_{i=1}^{n} |w_{i,t} - w_{i,t-1}|}_{\text{Transaction Penalty}} \tag{1}$$

where $w_{i,t}$ is the portfolio weight for asset i, $p_{i,t}$ is the asset price, and λ is the penalty factor. This formulation uses a penalty to discourage excessive trading, ensuring the agent only rebalances when the expected return outweighs the transaction costs.

3.4 Training Procedure

The historical dataset is divided chronologically into a training set (80%) and an out-of-sample evaluation set (20%). To facilitate iterative experimentation, training hyperparameters and environment variables (such as selected assets and window size) are managed via a centralized configuration file. This modular setup allows for systematic tuning of key parameters across runs, including batch size, rollout length (`n_steps`), learning rate, LSTM architecture (hidden size and recurrent layers), optimization epochs, and total timesteps.

Finally, training progress and learning dynamics, such as portfolio returns, allocation weights, and transaction penalties, are monitored using `TensorBoard`.

3.5 Evaluation and Preliminary Results

The learned policy is evaluated through a backtesting procedure on the out-of-sample test dataset (comprising approximately 750 trading days). The performance of the reinforcement learning agent is compared against a benchmark strategy consisting of a simple *buy-and-hold* portfolio.

Preliminary results indicate that the agent progressively learns non-trivial allocation strategies and dynamically adjusts portfolio weights according to market conditions, generating positive yield. The evolution of portfolio weights during the test phase illustrates how the agent adapts its allocation decisions. Thanks to the transaction penalty, the shifts across the 11 assets are purposeful and stable, proving the agent avoids pure speculation. Furthermore, as evidenced in Figure 2, the agent maintains a steady upward trajectory and successfully outperforms the buy-and-hold baseline in cumulative returns, demonstrating an ability to capture market upside while strategically managing exposure.

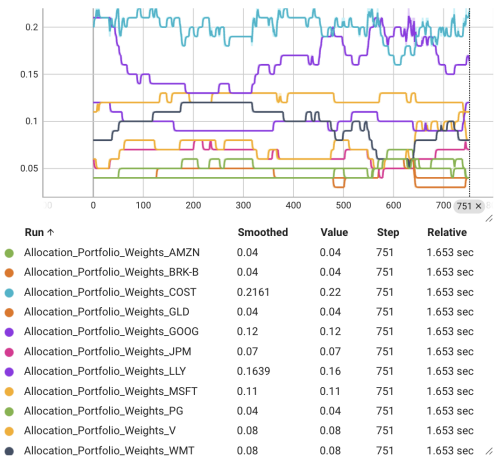


Figure 1: Dynamic portfolio allocation.

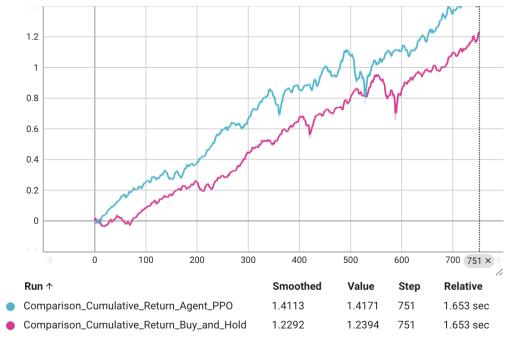


Figure 2: Cumulative return.

3.6 Future Directions

Moving forward, we plan to enhance the system's performance and robustness through three primary avenues. First, we intend to explore **alternative Deep Reinforcement Learning (DRL) algorithms** to compare their stability and convergence against our current PPO baseline. Second, we will focus on **feature engineering**, incorporating stationary technical indicators and risk-adjusted metrics into the observation space to provide the agent with richer market context. Finally, we will conduct systematic **hyperparameter tuning** of the model architecture and training process to optimize for long-term Sharpe ratios and more effective risk management.